

METHOD OVERLOADING AND METHOD OVERRIDING

Aim:

To understand and implement method overloading and method overriding.

PRE LAB EXERCISE

QUESTIONS

✓ Method Overloading:

Same method name, different parameters (same class).

→ Compile-time polymorphism.

✓ Method Overriding:

Same method name and parameters (parent-child).

→ Runtime polymorphism.

✓ Difference:

Overloading → Different parameters, same class, compile-time.

Overriding → Same parameters, inheritance, runtime.

IN LAB EXERCISE

Objective:

To demonstrate compile-time and runtime polymorphism.

PROGRAMS:

1.Student Result System (Method Overriding)

Description:

- Base class Student has method displayResult().
- Subclasses UGStudent and PGStudent override the method to show different grading systems.

Code :

```
import java.util.Scanner;
```

```
// Base class
```

```
class Student {  
    String name;  
  
    void displayResult() {  
        System.out.println("Student Result");  
    }  
}  
  
// UG Student subclass  
class UGStudent extends Student {  
    int marks;  
  
    UGStudent(String n, int m) {  
        name = n;  
        marks = m;  
    }  
  
    @Override  
    void displayResult() {  
        double percentage = (marks / 100.0) * 100;  
        System.out.println("UG Student: " + name);  
        System.out.println("Marks: " + marks);  
        System.out.println("Percentage: " + percentage + "%");  
    }  
}  
  
// PG Student subclass  
class PGStudent extends Student {
```

```

double gpa;

PGStudent(String n, double g) {
    name = n;
    gpa = g;
}

@Override
void displayResult() {
    System.out.println("PG Student: " + name);
    System.out.println("GPA: " + gpa + " / 10");
}
}

// Main class
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Input for UG student
        System.out.print("Enter UG Student Name: ");
        String ugName = sc.nextLine();
        System.out.print("Enter UG Student Marks (out of 100): ");
        int ugMarks = sc.nextInt();
        sc.nextLine(); // consume newline

        // Input for PG student
        System.out.print("Enter PG Student Name: ");

```

```
String pgName = sc.nextLine();

System.out.print("Enter PG Student GPA (0-10): ");

double pgGpa = sc.nextDouble();


// Create objects

Student s1 = new UGStudent(ugName, ugMarks);

Student s2 = new PGStudent(pgName, pgGpa);


System.out.println("\n--- Student Results ---");

s1.displayResult();

System.out.println();

s2.displayResult();

sc.close();

}

}
```

OUTPUT:

Sample Input:

Enter UG Student Name: Ram

Enter UG Student Marks (out of 100): 85

Enter PG Student Name: Ravi

Enter PG Student GPA (0-10): 9.2

Output:

--- Student Results ---

UG Student: Ram

Marks: 85

Percentage: 85.0%

PG Student: Ravi

GPA: 9.2 / 10

```
Main.java 1 X
1  import java.util.Scanner;    Main.java is a non-project file, only syntax errors are reported
2
3  // Base class
4  class Student {
5      String name;
6
7      void displayResult() {
8          System.out.println(x: "Student Result");
9      }
10 }
11
12 // UG Student subclass
13 class UGStudent extends Student {
14     int marks;
15
16     UGStudent(String n, int m) {
17         name = n;
18         marks = m;
19     }
20
21     @Override
22     void displayResult() {
23         double percentage = (marks / 100.0) * 100;
24         System.out.println("UG Student: " + name);
25         System.out.println("Marks: " + marks);
26         System.out.println("Percentage: " + percentage + "%");
27     }
28 }
29
30 PROBLEMS 1 OUTPUT DEBUG CONSOLE EXPLORER TERMINAL PORTS
31
32 Enter PG Student GPA (0-10): 9
33
34 --- Student Results ---
35 UG Student: Ranjith
36 Marks: 100
37 Percentage: 100.0%
38
39 PG Student: Santhosh
40 GPA: 9.0 / 10
41 PS C:\Users\ragur>
```

2. Calculator Program (Method Overloading)

Description:

Create a Calculator class with multiple add() methods to calculate:

- Addition of 2 integers
- Addition of 3 integers
- Addition of 2 double numbers

Code:

```
import java.util.Scanner;

class Calculator {

    int add(int a, int b) {

        return a + b;

    }

    int add(int a, int b, int c) {
```

```

        return a + b + c;
    }

    double add(double a, double b) {
        return a + b;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Calculator calc = new Calculator();

        System.out.print("Enter two integers: ");
        int x = sc.nextInt();
        int y = sc.nextInt();
        System.out.println("Sum of two integers: " + calc.add(x, y));

        System.out.print("Enter three integers: ");
        int p = sc.nextInt();
        int q = sc.nextInt();
        int r = sc.nextInt();
        System.out.println("Sum of three integers: " + calc.add(p, q, r));

        System.out.print("Enter two decimal numbers: ");
        double a = sc.nextDouble();
        double b = sc.nextDouble();
        System.out.println("Sum of two doubles: " + calc.add(a, b));
    }
}

```

```

        sc.close();
    }
}

```

Output:

Sample Input:

Enter two integers: 10 20

Enter three integers: 5 10 15

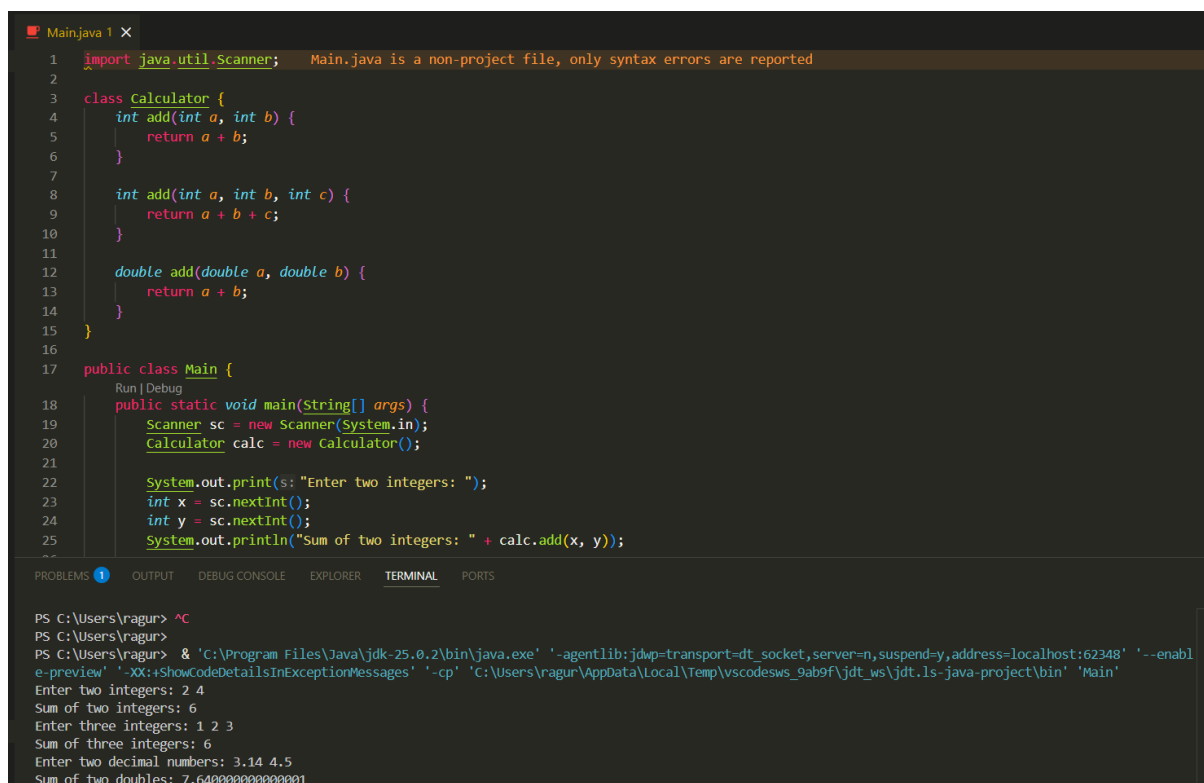
Enter two decimal numbers: 2.5 3.5

Output:

Sum of two integers: 30

Sum of three integers: 30

Sum of two doubles: 6.0



The screenshot shows a VS Code editor with a file named 'Main.java'. The code defines a 'Calculator' class with three methods: 'add' for two integers, 'add' for three integers, and 'add' for two doubles. A 'Main' class contains the 'main' method, which uses a 'Scanner' to take user input and calls the 'Calculator' methods to calculate the sums. The terminal output shows the program being run from the command line, followed by the user's input and the program's output: 'Sum of two integers: 6', 'Sum of three integers: 6', and 'Sum of two doubles: 7.640000000000001'.

```

1  import java.util.Scanner;
2
3  class Calculator {
4      int add(int a, int b) {
5          return a + b;
6      }
7
8      int add(int a, int b, int c) {
9          return a + b + c;
10     }
11
12     double add(double a, double b) {
13         return a + b;
14     }
15 }
16
17 public class Main {
18     public static void main(String[] args) {
19         Scanner sc = new Scanner(System.in);
20         Calculator calc = new Calculator();
21
22         System.out.print("Enter two integers: ");
23         int x = sc.nextInt();
24         int y = sc.nextInt();
25         System.out.println("Sum of two integers: " + calc.add(x, y));
26     }
27 }

```

```

PS C:\Users\ragur> ^C
PS C:\Users\ragur>
PS C:\Users\ragur> & 'C:\Program Files\Java\jdk-25.0.2\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:62348' '-enabl
e-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\ragur\AppData\Local\Temp\vscodesws_9ab9f\jdt_ws\jdt.ls-java-project\bin' 'Main'
Enter two integers: 2 4
Sum of two integers: 6
Enter three integers: 1 2 3
Sum of three integers: 6
Enter two decimal numbers: 3.14 4.5
Sum of two doubles: 7.640000000000001

```

POST LAB EXERCISE

✓ Is return type important?

- **Overloading:** Not important; parameters must be different.
- **Overriding:** Must be the same or covariant.

✓ Can you overload by changing only the return type?

No. The parameter list must be different.

✓ Can static methods be overridden or overloaded?

- **Overridden:** No (only method hiding).
- **Overloaded:** Yes.

✓ Can a method be overridden with different parameters?

No. Different parameters mean overloading, not overriding.

Result:

Thus the method overloading and overriding concepts were implemented and executed successfully.

ASSESSMENT

Description	Max Marks	Marks Awarded
Pre Lab Exercise	5	
In Lab Exercise	10	
Post Lab Exercise	5	
Viva	10	
Total	30	
Faculty Signature		